

Guia de API: Criar campo de relacionamento

Referência do fluxo de criação de campo **RELATIONSHIP** no LowCodeJS: as cardinalidades, o objeto **relationship**, os payloads e respostas reais do **POST /tables/:slug/fields**, e o **PUT** que muda a cardinalidade.

Base URL dos exemplos: **http://localhost:3000**. Os exemplos usam **curl** com o cookie de sessão gravado no login (**-b cookies.txt**). Troque os IDs pelos que o backend devolver.

Sumário

1. [Cardinalidades: 1:1, 1:N, N:N](#)
2. [O objeto relationship \(anatomia\)](#)
3. [Criar campo \(POST\)](#)
4. [Editar e mudar a cardinalidade \(PUT\)](#)
5. [Erros](#)

1. Cardinalidades: 1:1, 1:N, N:N

A cardinalidade não é um campo que você escolhe direto. Ela é **derivada** de dois flags: **field.multiple** (o lado source) e **relationship.mirror.multiple** (o lado espelho). O mesmo par decide onde o vínculo é gravado.

Relação	field.multiple	mirror.multiple	Armazenamento
1:1	não	não	FK na row do source (OWNS_FK)
1:N	sim	não	FK na row do lado N (OWNS_FK)
N:N	sim	sim	collection pivô relationship-links (PIVOT)

OWNS_FK grava a FK na própria row; o lado oposto (**REVERSE**) não grava nada e resolve por query reversa; **PIVOT** é o N:N via collection **relationship-links**. Nada disso é persistido como "cardinalidade": o backend recalcula a partir dos flags **multiple**. Você define a cardinalidade na criação escolhendo **multiple** e **mirror.multiple**, e a muda depois via **PUT** (seção 4).

2. O objeto relationship (anatomia)

O campo vira relacionamento quando **type: "RELATIONSHIP"** e traz o objeto **relationship**. A cardinalidade sai de **field.multiple**, e a obrigatoriedade de **field.required**, ambos fora do objeto **relationship**.

Chave	Tipo	Default	Significado
table	{ _id, slug }	n/a	tabela alvo (obrigatório)
field	{ _id, slug }	n/a	campo da tabela alvo usado como rótulo (obrigatório)

Chave	Tipo	Default	Significado
<code>order</code>	<code>"asc" "desc"</code>	<code>asc</code>	ordenação dos registros vinculados
<code>onDelete</code>	<code>"CASCADE" "SET_NULL" "RESTRICT"</code>	<code>null</code>	política ao excluir o alvo
<code>mirror</code>	<code>{ multiple, visible, label }</code>	<code>null</code>	config do lado espelho
<code>formMode</code>	<code>"select" "manage"</code>	<code>select</code>	multi-select simples ou tabela interna embutida
<code>max</code>	<code>number null</code>	<code>null</code>	limite de vínculos deste lado
<code>visible</code>	<code>boolean</code>	<code>true</code>	mostra a tabela interna deste lado
<code>customLabel</code>	<code>boolean</code>	<code>false</code>	rótulo composto em vez de um único <code>field</code>
<code>labelParts</code>	<code>{ path, label? }[]</code>	<code>[]</code>	partes do rótulo composto
<code>labelSeparator</code>	<code>string</code>	<code>" - "</code>	separador entre as partes
<code>relationshipId</code>	<code>string null</code>	<code>null</code>	back-pointer para a definição; preenchido pelo backend
<code>side</code>	<code>"source" "target"</code>	<code>null</code>	lado deste campo; preenchido pelo backend

O `onDelete` rege a exclusão do alvo: `CASCADE` remove os registros que apontavam, `SET_NULL` mantém e limpa o vínculo, `RESTRICT` bloqueia enquanto houver vínculo.

Dentro de `mirror`: `multiple` (bool, false), `visible` (bool, true), `label` (string, opcional). Você não envia `relationshipId` nem `side`; o backend os preenche ao criar o campo, junto com o campo espelho e a `RelationshipDefinition`.

Detalhes que os exemplos da seção 3 deixam claros:

- **`mirror.label` é preenchido pelo backend.** Se você omitir `mirror.label`, o backend grava ali o **nome da tabela source**. No exemplo de A para B a resposta volta com `mirror.label: "Tabela A"` mesmo o request não tendo mandado nada.
- **`formMode` é derivado, não uma escolha livre.** O backend usa `manage` apenas quando `field.multiple` e `mirror.multiple` são ambos `true` (o caso N:N); em qualquer outro caso, `select`.
- **`side` e `relationshipId` sempre voltam preenchidos.** Ao criar o campo o backend materializa o campo espelho na tabela alvo (`side: "target"`) e a definição, e grava o mesmo `relationshipId` nos dois lados.

3. Criar campo (POST)

`POST /tables/:slug/fields`

Os exemplos abaixo são capturas reais do fluxo. Uma tabela source **tabela-a** ganha dois campos de relacionamento: um 1:1 para **tabela-b** e um N:N para **tabela-c**. Cada bloco traz o request cru, a response 201 crua e o que o backend derivou.

3.1 Relacionamento 1:1 (**multiple: false, formMode: select**)

POST /tables/tabela-a/fields

Request. Note que **mirror** vai sem **label** e o **formMode** vem como **select**:

```
{
  "name": "Relacionamento com B",
  "slug": "relacionamento-com-b",
  "type": "RELATIONSHIP",
  "required": false,
  "multiple": false,
  "showInFilter": true,
  "tip": null,
  "label": null,
  "format": null,
  "defaultValue": null,
  "permissions": {
    "list": { "kind": "PUBLIC", "group": null },
    "form": { "kind": "PUBLIC", "group": null },
    "detail": { "kind": "PUBLIC", "group": null }
  },
  "widthInForm": 50,
  "widthInList": 10,
  "validations": [],
  "dropdown": [],
  "allowCustomDropdownOptions": false,
  "allowCreateRelationshipRecords": false,
  "relationship": {
    "table": { "_id": "6a4c789a57189679d9fc8fa0", "slug": "tabela-b" },
    "field": { "_id": "6a4c789a57189679d9fc8f9c", "slug": "nome" },
    "order": "asc",
    "customLabel": false,
    "labelParts": [],
    "labelSeparator": " - ",
    "visible": true,
    "onDelete": "SET_NULL",
    "mirror": { "multiple": false, "visible": false },
    "formMode": "select",
    "max": null
  },
  "category": [],
  "htmlContent": null,
  "group": null
}
```

Response 201. O backend cravou `mirror.label: "Tabela A"` (nome da tabela source), `relationshipId, side: "source"`, e os defaults do campo (`widthInDetail: 50`, `locked`, `native`, `trashed`, `trashedAt`):

```
{
  "_id": "6a4c793e57189679d9fc91d0",
  "name": "Relacionamento com B",
  "slug": "relacionamento-com-b",
  "type": "RELATIONSHIP",
  "required": false,
  "multiple": false,
  "showInFilter": true,
  "widthInForm": 50,
  "widthInList": 10,
  "widthInDetail": 50,
  "tip": null,
  "locked": false,
  "native": false,
  "label": null,
  "format": null,
  "defaultValue": null,
  "dropdown": [],
  "allowCustomDropdownOptions": false,
  "allowCreateRelationshipRecords": false,
  "relationship": {
    "table": { "_id": "6a4c789a57189679d9fc8fa0", "slug": "tabela-b" },
    "field": { "_id": "6a4c789a57189679d9fc8f9c", "slug": "nome" },
    "order": "asc",
    "customLabel": false,
    "labelParts": [],
    "labelSeparator": " - ",
    "visible": true,
    "onDelete": "SET_NULL",
    "mirror": { "multiple": false, "visible": false, "label": "Tabela A" },
    "relationshipId": "6a4c793e57189679d9fc91dd",
    "formMode": "select",
    "side": "source",
    "max": null
  },
  "category": [],
  "group": null,
  "validations": [],
  "htmlContent": null,
  "trashed": false,
  "trashedAt": null,
  "createdAt": "2026-07-07T03:57:50.697Z",
  "updatedAt": "2026-07-07T03:57:50.718Z"
}
```

Como `field.multiple` e `mirror.multiple` são ambos `false`, a cardinalidade é 1:1 e a FK mora na row do source. O `relationshipId` é `6a4c793e57189679d9fc91dd`.

3.2 Relacionamento N:N (`multiple: true + mirror.multiple: true, formMode: manage`)

POST /tables/tabela-a/fields

Request. Agora `multiple: true, mirror.multiple: true` e `mirror.visible: true`, e o `formMode` já vai como `manage`:

```
{
  "name": "Relacionamento com C",
  "slug": "relacionamento-com-c",
  "type": "RELATIONSHIP",
  "required": false,
  "multiple": true,
  "showInFilter": true,
  "tip": null,
  "label": null,
  "format": null,
  "defaultValue": null,
  "permissions": {
    "list": { "kind": "PUBLIC", "group": null },
    "form": { "kind": "PUBLIC", "group": null },
    "detail": { "kind": "PUBLIC", "group": null }
  },
  "widthInForm": 50,
  "widthInList": 10,
  "validations": [],
  "dropdown": [],
  "allowCustomDropdownOptions": false,
  "allowCreateRelationshipRecords": false,
  "relationship": {
    "table": { "_id": "6a4c78c457189679d9fc90a1", "slug": "tabela-c" },
    "field": { "_id": "6a4c78c457189679d9fc909d", "slug": "nome" },
    "order": "asc",
    "customLabel": false,
    "labelParts": [],
    "labelSeparator": " - ",
    "visible": true,
    "onDelete": "SET_NULL",
    "mirror": { "multiple": true, "visible": true },
    "formMode": "manage",
    "max": null
  },
  "category": [],
  "htmlContent": null,
  "group": null
}
```

Response 201:

```

{
  "_id": "6a4c79b557189679d9fc92a3",
  "name": "Relacionamento com C",
  "slug": "relacionamento-com-c",
  "type": "RELATIONSHIP",
  "required": false,
  "multiple": true,
  "showInFilter": true,
  "widthInForm": 50,
  "widthInList": 10,
  "widthInDetail": 50,
  "tip": null,
  "locked": false,
  "native": false,
  "label": null,
  "format": null,
  "defaultValue": null,
  "dropdown": [],
  "allowCustomDropdownOptions": false,
  "allowCreateRelationshipRecords": false,
  "relationship": {
    "table": { "_id": "6a4c78c457189679d9fc90a1", "slug": "tabela-c" },
    "field": { "_id": "6a4c78c457189679d9fc909d", "slug": "nome" },
    "order": "asc",
    "customLabel": false,
    "labelParts": [],
    "labelSeparator": " - ",
    "visible": true,
    "onDelete": "SET_NULL",
    "mirror": { "multiple": true, "visible": true, "label": "Tabela A" },
    "relationshipId": "6a4c79b557189679d9fc92b0",
    "formMode": "manage",
    "side": "source",
    "max": null
  },
  "category": [],
  "group": null,
  "validations": [],
  "htmlContent": null,
  "trashed": false,
  "trashedAt": null,
  "createdAt": "2026-07-07T03:59:49.287Z",
  "updatedAt": "2026-07-07T03:59:49.307Z"
}

```

Com os dois lados múltiplos a cardinalidade é N:N, o armazenamento passa a ser a collection pivô `relationship-links`, e o `formMode: manage` embute a tabela interna no formulário. O `relationshipId` é `6a4c79b557189679d9fc92b0`.

3.3 O campo espelho no lado alvo (`side: "target"`)

Ao criar o campo source o backend materializa o campo espelho na tabela alvo. Ele não aparece na response do POST. Você o lê buscando a outra tabela:

GET /tables/tabela-b

Dentro de `fields[]` vem o espelho gerado pelo backend. A estrutura (o `_id` da `tabela-a` é ilustrativo, o `relationshipId` é o mesmo dos dois lados):

```
{
  "_id": "6a4c793e57189679d9fc91e5",
  "name": "Tabela A",
  "slug": "tabela-a",
  "type": "RELATIONSHIP",
  "multiple": false,
  "relationship": {
    "table": { "_id": "<id-tabela-a>", "slug": "tabela-a" },
    "field": { "_id": "6a4c789a57189679d9fc8f9c", "slug": "nome" },
    "mirror": { "multiple": false, "visible": true, "label":
"Relacionamento com B" },
    "relationshipId": "6a4c793e57189679d9fc91dd",
    "side": "target"
  }
}
```

O `name` do espelho é o nome da tabela source ("Tabela A") porque nenhum `mirror.label` foi enviado. O `relationshipId` casa com o do campo source, e o `side` aqui é "target".

4. Editar e mudar a cardinalidade (PUT)

PUT /tables/tabela-a/fields/6a4c793e57189679d9fc91d0

Passar o "Relacionamento com B" de 1:1 para 1:N é só flipar `multiple` para `true` no campo source. O PUT manda o objeto do campo inteiro (incluindo `slug`, `trashed`, `trashedAt`) e o `relationship` completo com o `mirror`:

```
{
  "name": "Relacionamento com B",
  "slug": "relacionamento-com-b",
  "type": "RELATIONSHIP",
  "required": false,
  "multiple": true,
  "showInFilter": true,
  "tip": null,
  "label": null,
  "format": null,
  "defaultValue": null,
  "permissions": {
    "list": { "kind": "PUBLIC", "group": null },
    "form": { "kind": "PUBLIC", "group": null },
    "detail": { "kind": "PUBLIC", "group": null }
  }
}
```

```

},
"widthInForm": 50,
"widthInList": 10,
"validations": [],
"dropdown": [],
"allowCustomDropdownOptions": false,
"allowCreateRelationshipRecords": false,
"relationship": {
  "table": { "_id": "6a4c789a57189679d9fc8fa0", "slug": "tabela-b" },
  "field": { "_id": "6a4c789a57189679d9fc8f9c", "slug": "nome" },
  "order": "asc",
  "customLabel": false,
  "labelParts": [],
  "labelSeparator": " - ",
  "visible": true,
  "onDelete": "SET_NULL",
  "mirror": { "multiple": false, "visible": false, "label": "Tabela A" },
  "formMode": "select",
  "max": null
},
"category": [],
"htmlContent": null,
"trashed": false,
"trashedAt": null
}

```

Response 200. O campo agora é `multiple: true` (1:1 virou 1:N); o `mirror` continua `multiple: false`:

```

{
  "_id": "6a4c793e57189679d9fc91d0",
  "name": "Relacionamento com B",
  "slug": "relacionamento-com-b",
  "type": "RELATIONSHIP",
  "required": false,
  "multiple": true,
  "showInFilter": true,
  "widthInForm": 50,
  "widthInList": 10,
  "widthInDetail": 50,
  "tip": null,
  "locked": false,
  "native": false,
  "label": null,
  "format": null,
  "defaultValue": null,
  "dropdown": [],
  "allowCustomDropdownOptions": false,
  "allowCreateRelationshipRecords": false,
  "relationship": {
    "table": { "_id": "6a4c789a57189679d9fc8fa0", "slug": "tabela-b" },
    "field": { "_id": "6a4c789a57189679d9fc8f9c", "slug": "nome" },
    "order": "asc",

```



```

    "customLabel": false,
    "labelParts": [],
    "labelSeparator": " - ",
    "visible": true,
    "onDelete": "SET_NULL",
    "mirror": { "multiple": false, "visible": false, "label": "Tabela A" },
    "relationshipId": "6a4c793e57189679d9fc91dd",
    "formMode": "select",
    "side": "source",
    "max": null
  },
  "category": [],
  "group": null,
  "validations": [],
  "htmlContent": null,
  "trashed": false,
  "trashedAt": null,
  "createdAt": "2026-07-07T03:57:50.697Z",
  "updatedAt": "2026-07-07T04:01:13.358Z"
}

```

Atenção ao editar um RELATIONSHIP. O backend sobrescreve o objeto `relationship` inteiro no update. Reenvie-o **completo, incluindo `mirror`**. Se você mandar `relationship` sem `mirror`, o backend re-sincroniza o espelho como `mirror.multiple: false`, e você perde a configuração do outro lado. É por isso que o request acima carrega o `mirror` mesmo só querendo mudar `multiple`.

5. Erros

Do **POST** e do **PUT** de campo de relacionamento:

- **404 TABLE_NOT_FOUND**: a tabela alvo (`relationship.table`) não existe. Crie-a antes de criar o campo.
- **409 FIELD_ALREADY_EXIST**: já existe um campo com esse slug na tabela.

O envelope de erro:

```

{
  "message": "Mensagem legível em pt-BR",
  "code": 409,
  "cause": "FIELD_ALREADY_EXIST",
  "errors": {}
}

```

`code` é o status HTTP e `cause` é a string estável para o cliente ramificar.